

Practical - 8

Aim: Given a grid of dimension NXM where each cell in the grid can have values 0,1 or 2 which has the following meaning :

0: empty cell

1: cell have fresh oranges

2: cell have rotten oranges

We have to determine what is the min time required to rot all oranges. A rotten orange at index [i,j] can rot other fresh oranges at index [i-1,j],[i+1,j],[i,j-1],[i,j+1] (up, down , left, right) in unit time.

PROGRAM CODE :

```
from collections import deque
```

```
def min_time_to_rot_oranges(grid):
```

```
    # Dimensions of the grid
```

```
    N = len(grid)
```

```
    M = len(grid[0])
```

```
    # Directions for moving up, down, left, and right
```

```
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    # Queue to store rotten oranges with their coordinates and time to rot
```

```
    queue = deque()
```

```
    # Count of fresh oranges
```

```
    fresh_oranges = 0
```

```
    # Initialize the queue with all rotten oranges and count fresh oranges
```

```
    for i in range(N):
```

```
        for j in range(M):
```

```
            DAA(303105219)5B13/AI/CSE/PIET/
```

```
if grid[i][j] == 2:
    queue.append((i, j, 0)) # (row, col, time)
elif grid[i][j] == 1:
    fresh_oranges += 1

# If there are no fresh oranges to rot, return 0
if fresh_oranges == 0:
    return 0

# BFS to spread the rot
time_to_rot = 0
while queue:
    x, y, time = queue.popleft()

    # Explore the four directions
    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        # Check if the new position is within bounds and has a fresh orange
        if 0 <= nx < N and 0 <= ny < M and grid[nx][ny] == 1:
            # Rot the fresh orange
            grid[nx][ny] = 2
            fresh_oranges -= 1
            queue.append((nx, ny, time + 1))
            time_to_rot = time + 1

# If there are still fresh oranges left, return -1
if fresh_oranges > 0:
    return -1

return time_to_rot

# Function to take input for the grid
def input_grid():
    DAA(303105219)5B13/AI/CSE/PIET/
```

```
N, M = map(int, input("Enter the dimensions of the grid (N M): ").split())
```

```
grid = []
```

```
print("Enter the grid values (0 for empty, 1 for fresh, 2 for rotten):")
```

```
for i in range(N):
```

```
    row = list(map(int, input().split()))
```

```
    grid.append(row)
```

```
return grid
```

```
# Main function
```

```
if __name__ == "__main__":
```

```
    grid = input_grid()
```

```
    result = min_time_to_rot_oranges(grid)
```

```
    if result == -1:
```

```
        print("Not all oranges can rot.")
```

```
    else:
```

```
        print(f"Minimum time required to rot all oranges: {result}")
```

```
1  from collections import deque
2
3  def min_time_to_rot_oranges(grid):
4      # Dimensions of the grid
5      N = len(grid)
6      M = len(grid[0])
7
8      # Directions for moving up, down, left, and right
9      directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
10
11     # Queue to store rotten oranges with their coordinates and time to rot
12     queue = deque()
13
14     # Count of fresh oranges
15     fresh_oranges = 0
16
17     # Initialize the queue with all rotten oranges and count fresh oranges
18     for i in range(N):
19         for j in range(M):
20             if grid[i][j] == 2:
21                 queue.append((i, j, 0)) # (row, col, time)
22             elif grid[i][j] == 1:
23                 fresh_oranges += 1
24
25     if not queue:
26         return -1
27
28     time = 0
29     while queue:
30         size = len(queue)
31         for _ in range(size):
32             i, j, time = queue.popleft()
33             for di, dj in directions:
34                 ni, nj = i + di, j + dj
35                 if 0 <= ni < N and 0 <= nj < M and grid[ni][nj] == 1:
36                     grid[ni][nj] = 2
37                     queue.append((ni, nj, time + 1))
38         time += 1
39
40     if fresh_oranges == 0:
41         return time
42     else:
43         return -1
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\dspgy> & C:/Users/dspgy/AppData/Local/Programs/Python/Python311/python.exe "c:/wastage/8 Daa.py"
Enter the dimensions of the grid (N M): 3 3
Enter the grid values (0 for empty, 1 for fresh, 2 for rotten):
2 1 0
1 1 0
0 1 1
Minimum time required to rot all oranges: 4
```